DALARNA UNIVERSITY

Assignment
Extra Report

Authors
Ali Adrian Nasrat, Diana Dawidi, Tesnim Omran, Ibrahim Mizher

Group
4

Course code
GIK2PF

Date
07/01/26

1. Introduction

Service–Oriented Architecture (SOA) is a design paradigm where functionality is provided as loosely coupled, reusable services that communicate over standard interfaces. In this assignment, SoapUI was used as a testing tool to explore service interaction, service composition and the fundamental SOA roles.

The objective of this assignment was to:

- Explore REST-based web services using SoapUI
- Understand SOA concepts (service provider, requestor, registry)
- Implement a composition of services, where the output of one service is used as an input to another

The chosen scenario was Place → Weather, where a geographical place name is first resolved to latitude and longitude, which are then used to retrieve weather data.

2. Tools and Services Used

- SoapUI
  Used to create REST projects, test cases and property transfers

- Geocoding API (Open-Meteo)
  Endpoint: https://geocoding-api.open-meteo.com/v1/search

- Weather Forecast API (Open-Meteo)
  Endpoint: https://api.open-meteo.com/v1/forecast

These services are publicly available REST APIs and require no authentication.

3. SOA Concepts in Practice

The implemented solution demonstrates the three core SOA roles:

- Service Provider
  Open-Meteo provides the geocoding and weather services

- Service Requestor
  SoapUI acts as the client that send HTTP requests and consumes responses

- Service Registry
  While no formal Universal Description, Discovery and Integration (UDDI) registry is used, service documentation and endpoint URLs act as a lightweight registry mechanism

4. Service Composition Design

The composition follows three sequential steps within a SoapUI TestCase:

1. FindPlace – Resolve a city name to latitude and longitude
2. SaveLatLon – Extract and store coordinates as TestCase properties
3. GetWeather – Use stored coordinates to retrieve weather data

This demonstrates data-driven service composition, where one service's response dynamically feeds another service.

5. Implementation in SoapUI

5.1 FindPlace – Geocoding Request

This step requires the geocoding service using the city name (in this case Stockholm) and returns structured JSON data containing geographical coordinates.

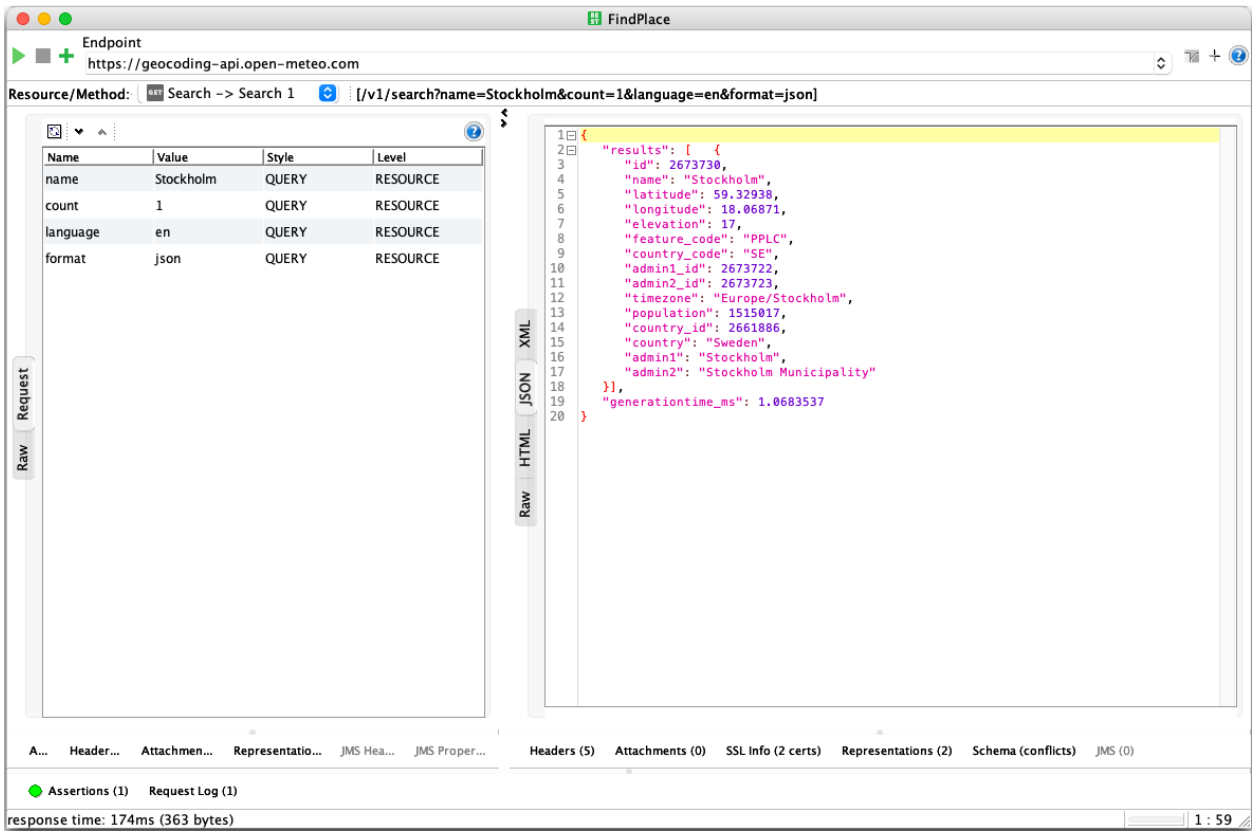

Figure 1: *FindPlace REST request retrieving latitude and longitude for Stockholm.*

This request sends a GET request to the Open-Meteo geocording API. The response contains a results array, from which latitude and longitude values are later extracted for service composition.

5.2 SaveLatLon – Property Transfer

The Property Transfer step extracts latitude and longitude from the JSON response using JSONPath expressions and stored them as TestCase properties.

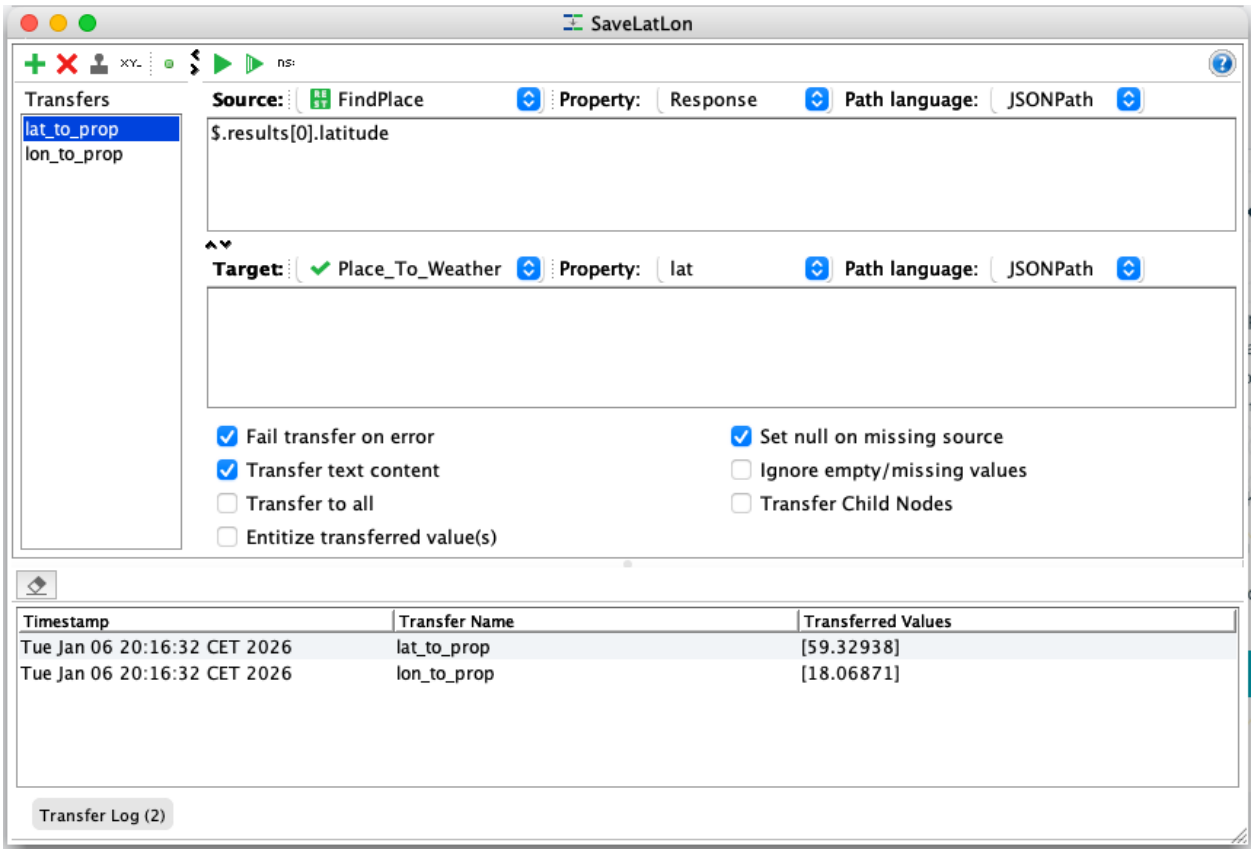

Figure 2: *Property Transfer step extracting latitude and longitude from the FindPlace response.*

JSONPath expressions ($.results[0].latitude and $.results[0].longitude inside each prop) are used to dynamically store values into SoapUI TestCase properties (lat and lon). These properties enable data reuse across service calls.

5.3 GetWeather – Weather Forecast Request

The final step retrieves current weather data using the coordinates obtained from the previous service.

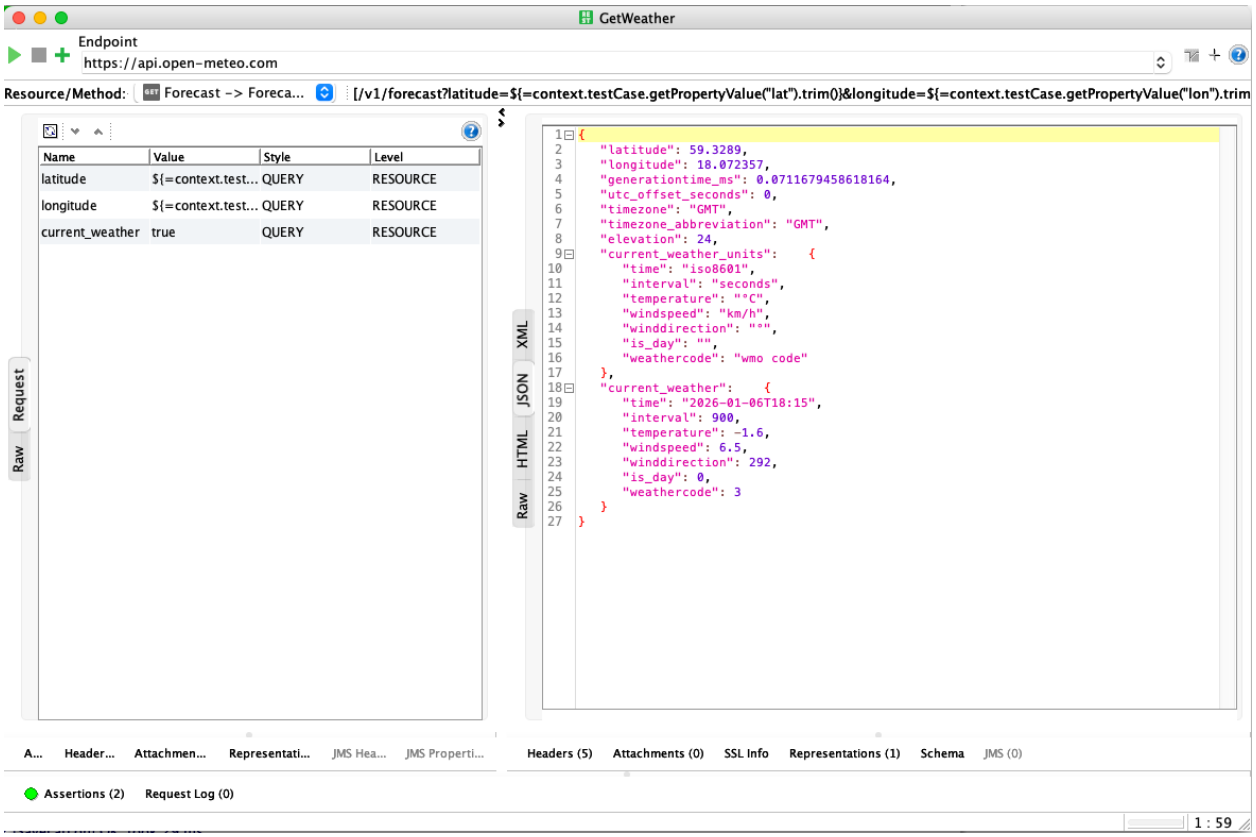

Figure 3: *GetWeather REST request using dynamic latitude and longitude properties.*

The weather request references the stored TestCase properties using expressions. This allows the request to adapt dynamically to any place provided in the first step, completing the service composition.

5.3.1 Example SoapUI Request Configuration (Excerpt)

```
<con:restRequest methodName="GET" endpoint="https://api.open-meteo.com/v1/forecast">
    <con:parameter name="latitude"
value="${=context.testCase.getPropertyValue("lat").trim()}" style="QUERY"/>
    <con:parameter name="longitude"
value="${=context.testCase.getPropertyValue("lon").trim()}" style="QUERY"/>
    <con:parameter name="current_weather" value="true" style="QUERY"/>
</con:restRequest>
```

This configuration illustrates how latitude and longitude values extracted from the geocoding service are reused as dynamic inputs in the weather service request, thereby enabling service composition in SoapUI.

6. Execution Result

The full TestCase execution completes successfully, demonstrating correct sequencing and data transfer between services.

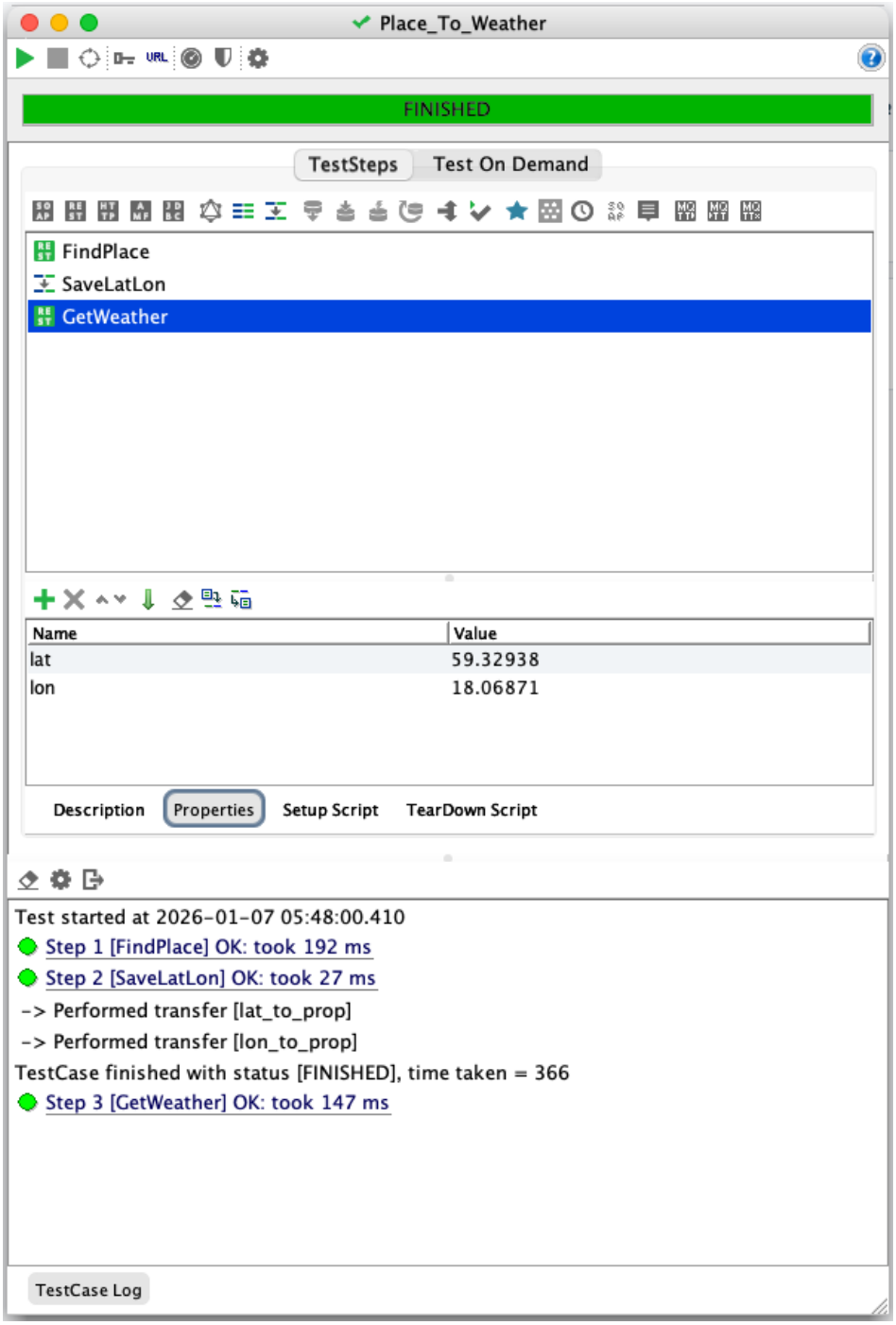

Figure 4: *Execution of the composed Place_To_Weather TestCase in SoapUI.*

This figure shows the successful execution of the composed TestCase in SoapUI. The workflow consists of three sequential steps: FindPlace, which retrieves geographic coordinates for a given location, SaveLatLon, which extracts and stores latitude and longitude as TestCase properties, and GetWeather, which uses these properties to request current weather data. The green status indicators confirm that all steps executed successfully and that the service composition functioned as intended.

7. Conclusion

In this assignment, SoapUI was successfully used to implement and test a REST-based service composition. A place name was resolved to geographical coordinates, which were then used to retrieve real-time weather data. The exercise reinforced key SOA principles and demonstrated how services can be achieved without custom code.